Conditionals & Repetitions - Python

WI1102 Computational Thinking Teaching Team

Institut Teknologi Bandung © 2026

Conditionals

Objectives

- Students can explain the definition and types of conditionals.
- Students can correctly use conditionals notation.
- Students can apply various types of conditionals to solve simple problems.

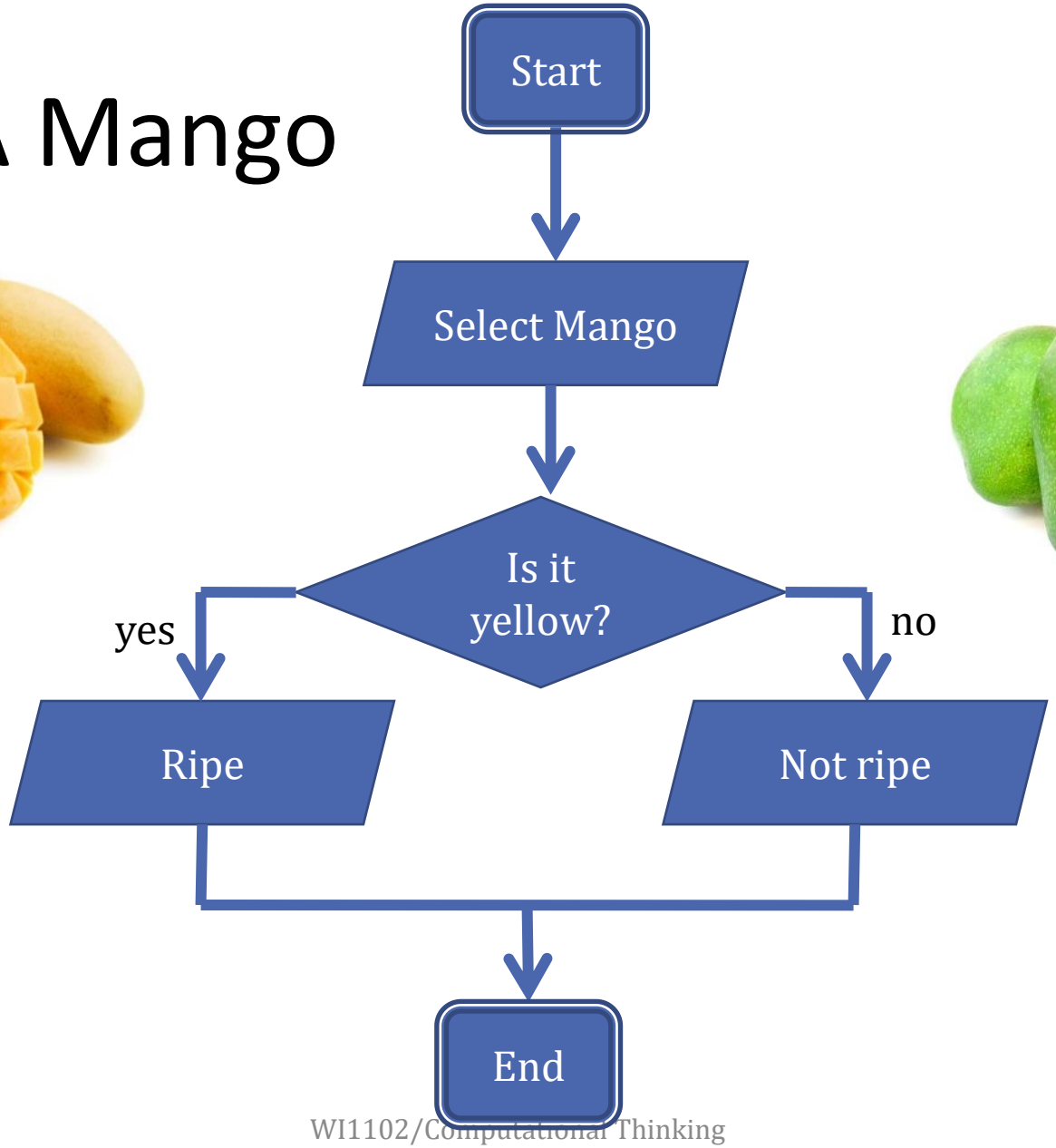
Example 1. Selecting A Mango

- Conditionals can be applied to everyday life—for example, when selecting a mango.
- A ripe mango ready to eat is yellow.
- If it is not yellow, it is not ripe.



Flowchart
Selecting A Mango





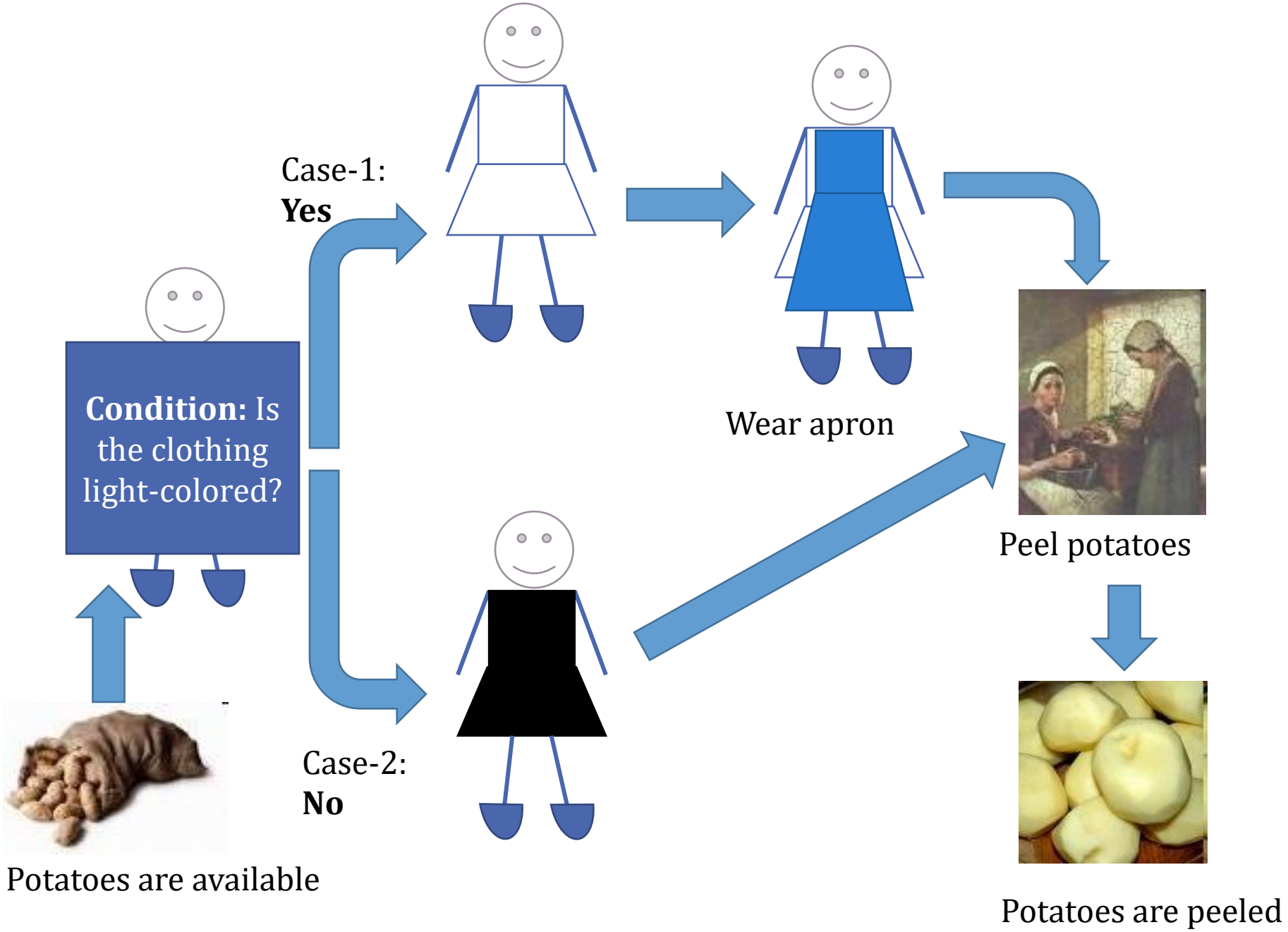


Selecting A Mango - Pseudocode

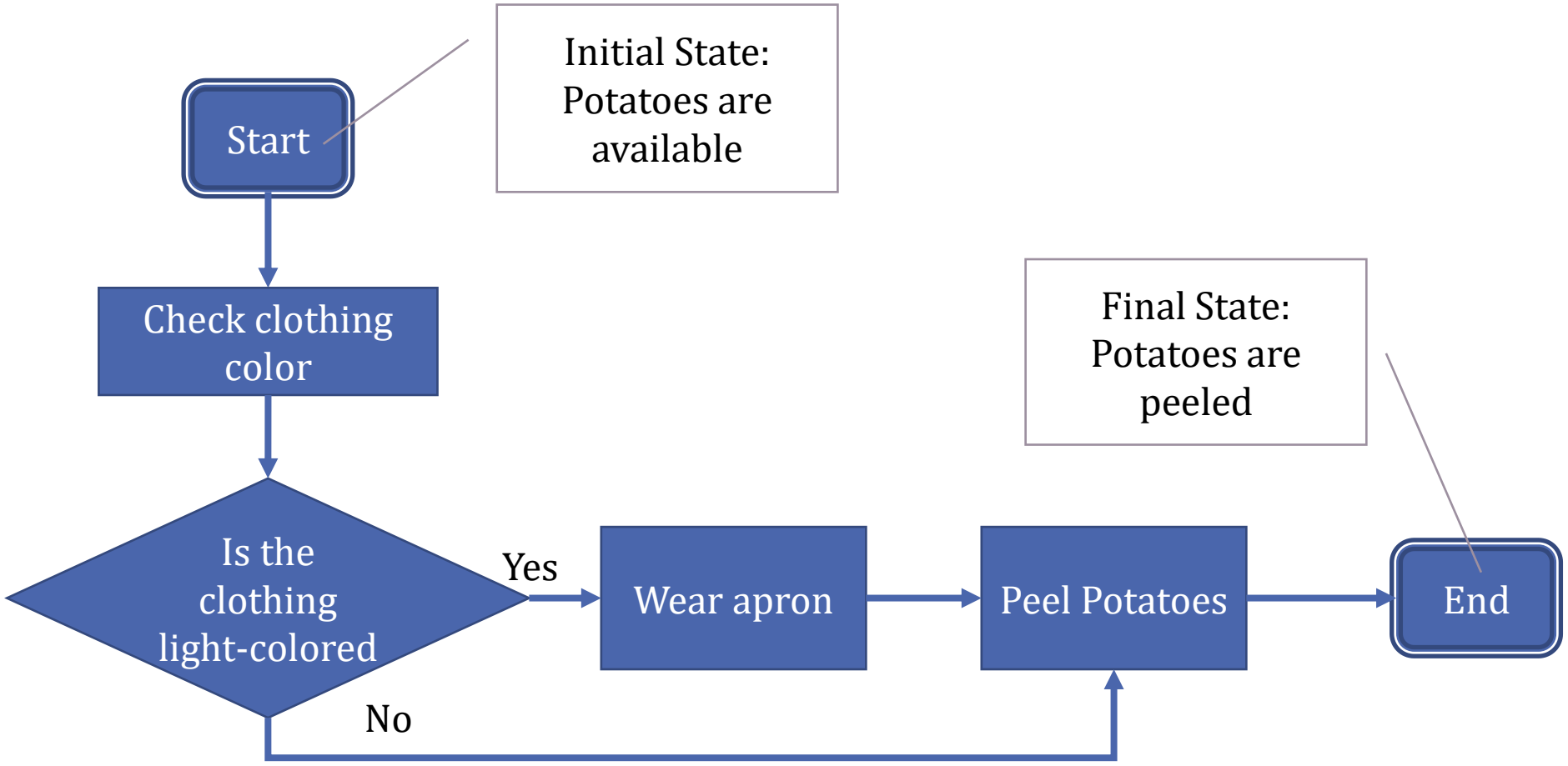
```
Select Mango
if (Is it yellow? = true) then
     Ripe
else { Is it yellow? = false }
     Not Ripe
```

Example 2. Preparing Potatoes for Dinner

- Every day, Mom prepares potatoes for dinner.
- Based on observation, there are days when Mom wears an apron while peeling potatoes, and other days when she doesn't.
  - It turns out that if Mom is wearing light-colored clothing, she wears an apron—fearing her clothes might look dirty.
  - If she isn't (i.e., wearing dark-colored clothing), she doesn't wear an apron.


Case-1:
Yes
Condition: Is the clothing light-colored?
Wear apron
Peel potatoes
Case-2:
No
Potatoes are available
Potatoes are peeled

Flowchart: Preparing Potatoes for Dinner



Preparing Potatoes for Dinner – Pseudocode

```
Check_clothing_color

if (Is_the_clothing_light_colored? = yes) then
        Wear_apron
{ else : Is_the_clothing_light_colored? = no,  do nothing }

Peel_potatoes
```

Conditionals (1)

- Allows us to write the same code that produces different executions
- Often referred to as **branching**
  - Involves a choice (branching) from a single step to multiple possible steps
- Consists of:
  - **Condition**: an expression that evaluates to true or false
  - **Action**: a statement executed if the condition associated with that action is met

Conditionals (2)

- Proper conditionals must satisfy two criteria:
  - **COMPLETE**: all conditions/cases are fully defined
  - **DISJOINT**: there is no overlap between conditions/cases
- Example: Given an integer, say A, determine whether the number is positive, negative, or zero.
  - There are three complete and disjoint cases:
    - A > 0
    - A < 0
    - A = 0
  - No other cases can be defined, and the three cases do not overlap.

General Syntax

Python

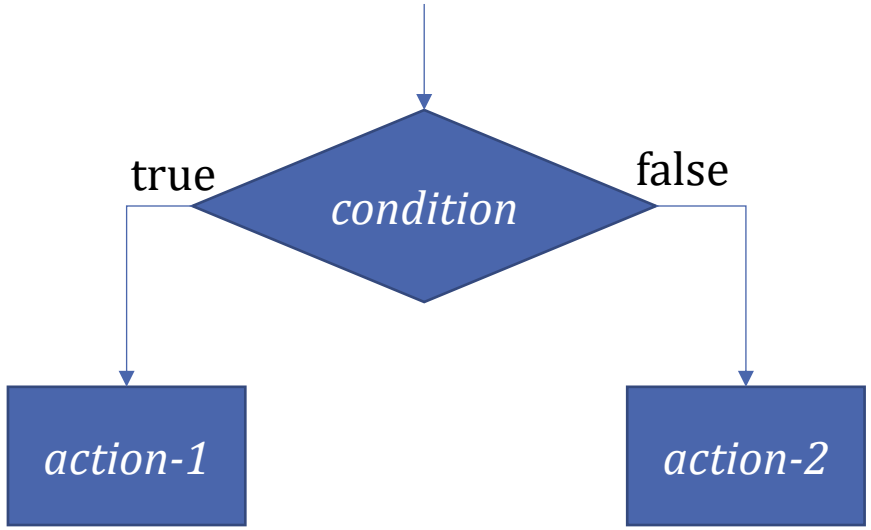
```
if ( condition ):
      action-1
else: # condition = false
      action-2
```

If action-1 or action-2 consists of more than one instructions, make sure the indentation is neat

Pseudocode

```
if ( condition ) then
     action-1
else { condition=false }
     action-2
```

Flowchart



Types of Conditionals (in Python)

One Case

```
if ( condition ):
       action-1

# if condition=false
# no action defined
```

Two [Complementary] Cases

```
if ( condition ):
     action-1
else: # condition=false
     action-2
```

Many Cases

```
if (condition-1):
       action-1
elif (condition-2):
       action-2
elif (...):
       # condition-3 ... dst
       ...
else: # condition-n
       action-n
```

Please adjust the pseudocodes and flowcharts, or check the following examples.

Example 3. Is the integer positive? [One Case]

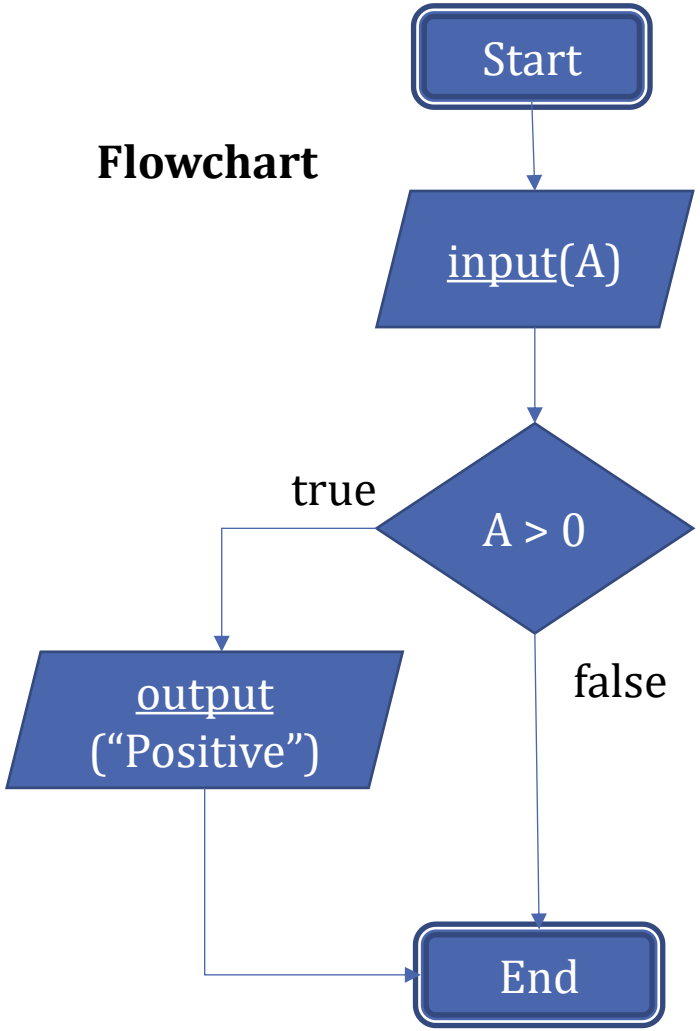
- Given an integer, for example A, state whether the integer is positive or not.
- Condition: **Is A > 0?**
- Cases:
  - If **yes**, then: write “Positive”
  - If **no**, do nothing.

Example 3. Pseudocode + Flowchart

Pseudocode

```
input(A)
if (A > 0) then
        output(“Positive”)
{ else: do nothing}
```

Flowchart



Example 3. Python

```
# Program Print_Positive
# Input A; If A >= 0, print "positive"

# DECLARATIONS
# A : int

# ALGORITHM
A = int(input())

if (A >= 0):
   print("positif")
# else: tidak dilakukan apa pun
```

Example 4. Odd or Even? [Two Complementary Cases]

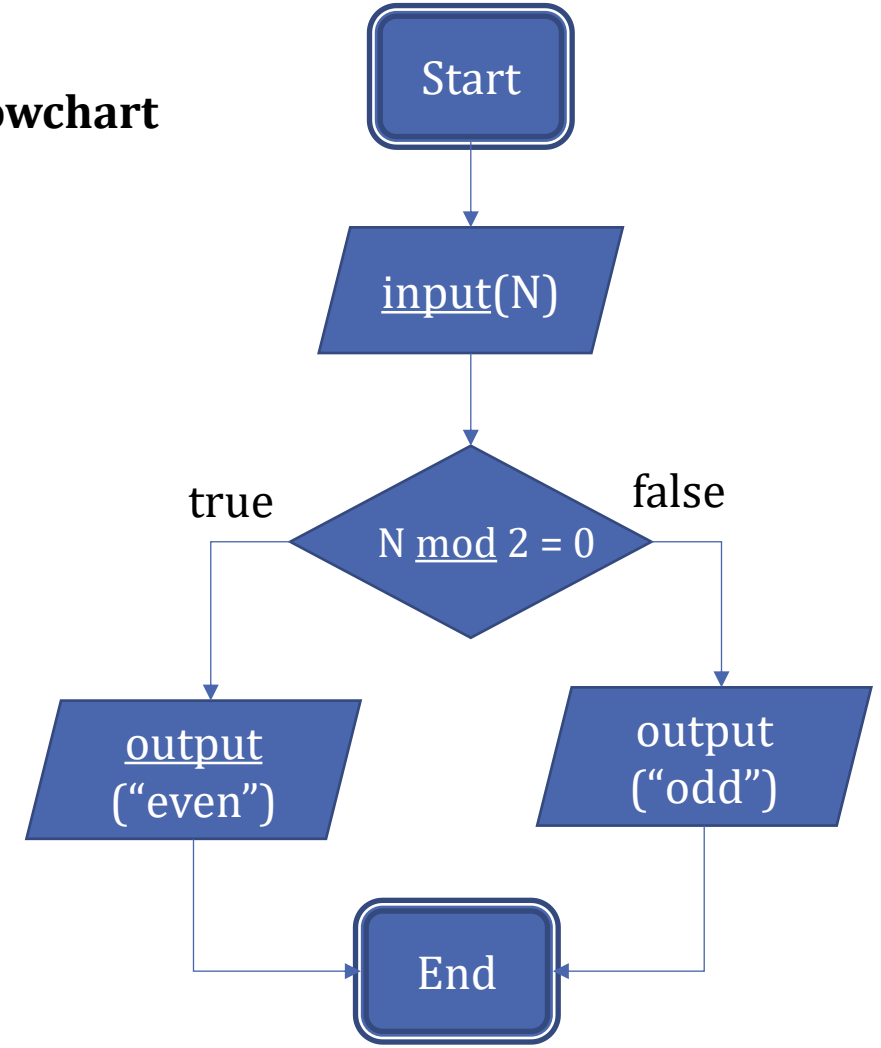
- Write a program that accepts a positive integer input (assume the input is valid), for example N, and then determines whether the number is even or odd.
- N is an even number if N mod 2 = 0; if N mod 2 = 1, then N is an odd number.
- There are no other cases.
- Cases:
  - If N mod 2 = 0, print “even”.
  - Otherwise (N mod 2 = 1), print “odd”.

Example 4. Pseudocode + Flowchart

Pseudocode

```
input(N)
if (N mod 2 = 0) then
        output("even")
else { N mod 2 = 1 }
        output("odd")
```

Flowchart



Example 4. Python

```
# Program Odd_Even
# Input N>0. If N is even, print "even"
# If not, print "odd"

# DECLARATIONS
# N : int

# ALGORITHM
N = int(input()) # Assume: N > 0

if (N % 2 == 0):
    print("even")
else: # N % 2 == 1
    print("odd")
```

Example 5. Positive, Negative, or Zero?

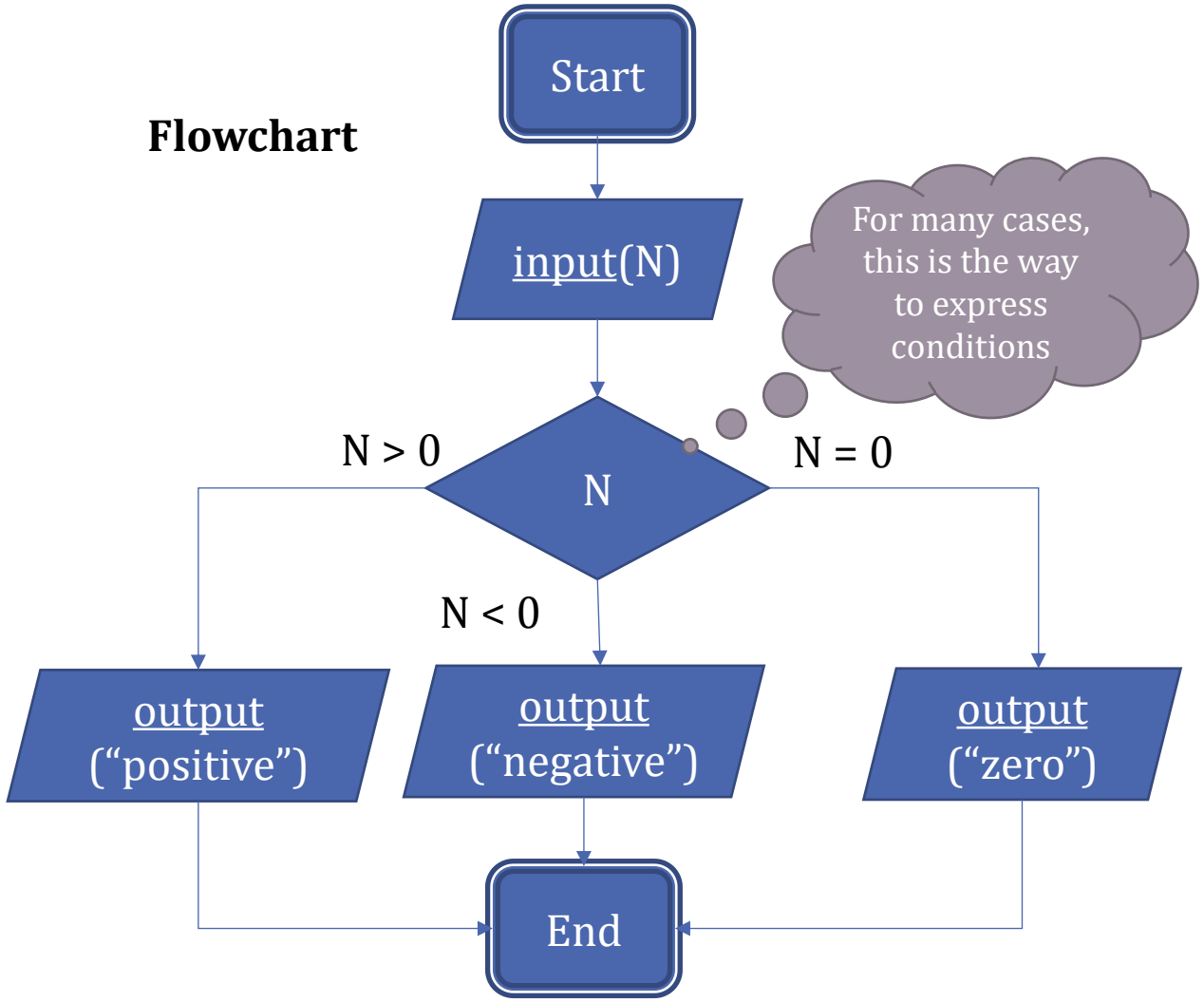
- Write a program that accepts an integer input, for example N, and determines whether N is a positive integer, a negative integer, or zero.
- Cases:
  - If N > 0, print “positive”
  - If N < 0, print “negative”
  - If N = 0, print “zero”

Example 5. Pseudocode + Flowchart

Pseudocode

```
input(N)
if (N > 0) then
      output(”positive”)
else if (N < 0) then
      output(“negative”)
else { N = 0 }
      output(“zero”)
```

Flowchart



Example 5. Python

```
# Program Print_Number
# Input N. Print whether N is positive, negative, or zero

# DECLARATIONS
# N : float

# ALGORITHM
N = int(input())

if (N > 0):
    print("positive")
elif (N < 0):
    print("negative")
else: # N = 0
    print("zero")
```

Repetition/Loop

Objectives

- Students can explain the types of loops, their usage, and their constituent elements.
- Students can correctly use the appropriate loop notation.
- Students can effectively utilize the various types of loops to solve assigned simple problems.

Preparing Potatoes for Dinner

- Assumption: There is an unlimited supply of potatoes.
- One day, Mom peeled only four potatoes because only family members were having dinner.
- On another occasion, Mom invited 400 WI1102 students to her home for dinner, so she peeled 400 potatoes for everyone.
- On yet another occasion, Mom did not know how many people would be having dinner.
  - After peeling each potato, she checked whether the quantity was sufficient according to the number of arriving guests.



Do we have enough peeled potatoes?
Number of potatoes = 4?
Number of potatoes = 400?
Number of potatoes = number of guests arriving



Peel 1 potato

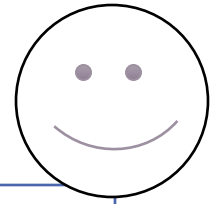


Number of peeled potatoes is enough

Print 1 and 2

- Write a program that prints the numbers 1 and 2 and then 1+2 to the screen
- Example output:

```
1
2
3
```



```
# ALGORITHM
print(1)
print(2)
print(1+2)
```

Print 1 to 3

- Write a program that prints the numbers 1 to 3 and then 1+2+3 to the screen
- Example output:

```
1
2
3
6
```



```
# ALGORITHM
print(1)
print(2)
print(3)
print(1+2+3)
```

Print 1 to 10

- Write a program that prints the numbers 1 to 10 and then 1+2+3+…+10 to the screen
- Example output:

```
1
2
3
4
5
6
7
8
9
10
55
```



```
# ALGORITHM
print(1)
print(2)
print(3)
print(4)
print(5)
print(6)
… #continue on your own!
print(10)
print(1+2+3+4+5+6+7+8+9+10)
```

Print 1 to 100

- Write a program that prints the numbers 1 to 100 and then 1+2+3+…+100 to the screen
- Example output:

```
1
2
3
4
5
6
7
8
9
10
… // continue!!
```



```
# ALGORITHM
print(1)
print(2)
print(3)
print(4)
print(5)
print(6)
… #continue on your own!!
print(100)
print(1+2+3+4+5+6+7+8+9+10+ … #continue!!)
```

How about…

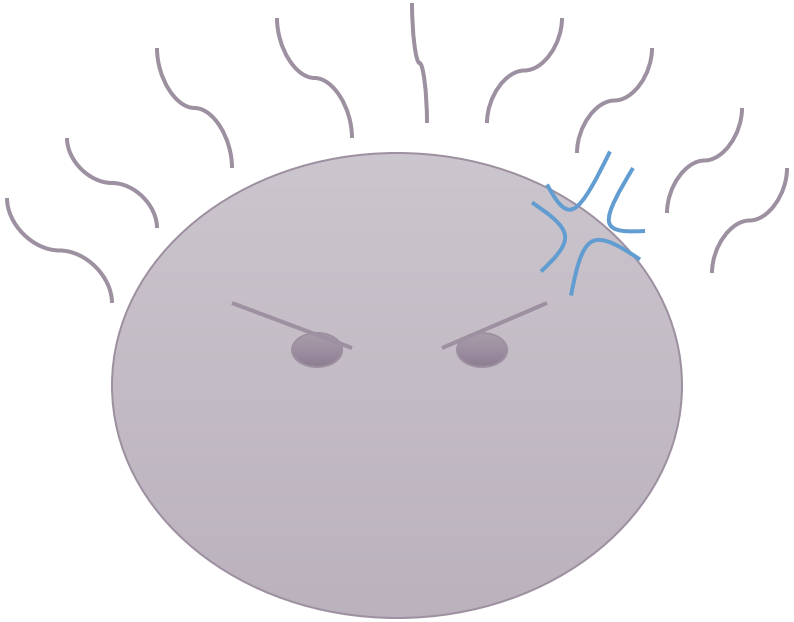
You have to print and add up

1 to 1000 ???

1 to 10000 ???

1 to 1000000 ???

….



Repetition: The Background

- Carrying out an instruction, even an action, repeatedly
  - Computer: has the same performance
  - Humans: have a tendency to make mistakes (because they are tired or bored)



Repetition (Looping)

- Elements:
  - **Repetition-condition**: logical expression
  - **Repetition-body**: action that is repeated
- Types of repetition notation in Python:
  - Based on counter: **for**
  - Based on the repetition condition at the beginning: **while**

Example 1

- Write a program that accepts as input an integer for example N and writes the numbers 1, 2, 3, ... N and writes 1+2+3+...+N to the screen.
- Assume N > 0.
- Examples:

N = **1**
Output on screen:
1
1

N = **5**
Output on screen:
1
2
3
4
5
15

N = **10**
Output on screen:
1
2
3
4
5
6
7
8
9
10
55

Looping Based on Counter (for)

- Repetition is carried out based on the price range of an counter variable (in the previous example i)
  - The price range for processed counters is from **hmin** to **hmax**
- The counter must be a variable with a type that defines its successor and predecessor, for example: integer
- The action will be carried out as long as the counter value is still within the specified range
- The counter price is incremented each time the action is performed
  - For this reason, the final value of the range must be written **hmax+**1 (so that hmax is still processed)

Looping Based on Counter (for)

Python

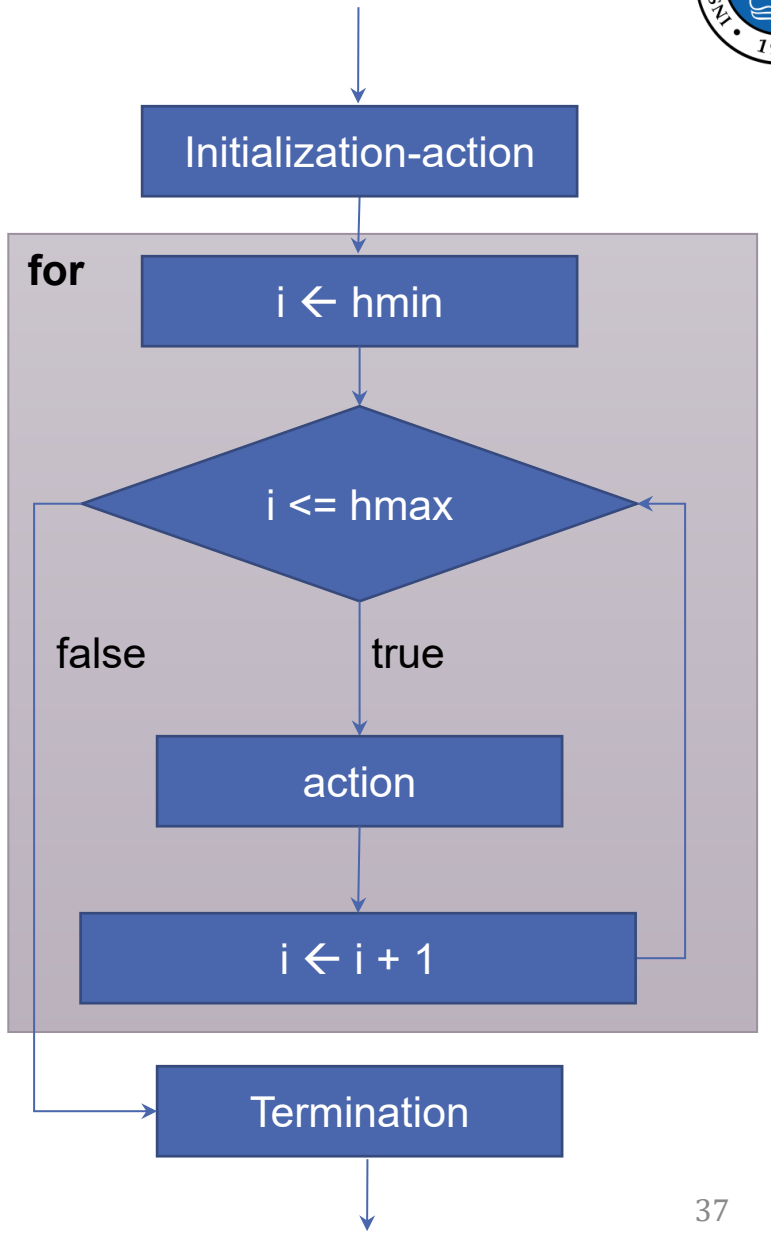
```
Initialization-action
for i in range(hmin,hmax+1):
    action
Termination
```

i is a counter variable (can be replaced by another variable)
hmin = value of i at the beginning of the loop; *hmax* = last i value processed;
The value of i when exiting the loop is hmax+1
Every time it repeats, i is incremented (plus 1)

pseudocode

```
Initialization-action
i traversal [hmin..hmax]
    action
Termination
```

flowchart



Example 1. for

```
# Program AddingUp_Numbers
# Calculate 1+2+3+...+N and print the result.
# Assumption: N > 0

# DECLARATIONS
# N : int
# i, sum : int

# ALGORITHM
N = int(input())      # initialization
sum = 0               # initialization

for i in range(1,N+1):
    print(i)          # action
    sum = sum + i     # action

print(sum)            # Termination
```

Counting Backward

Python

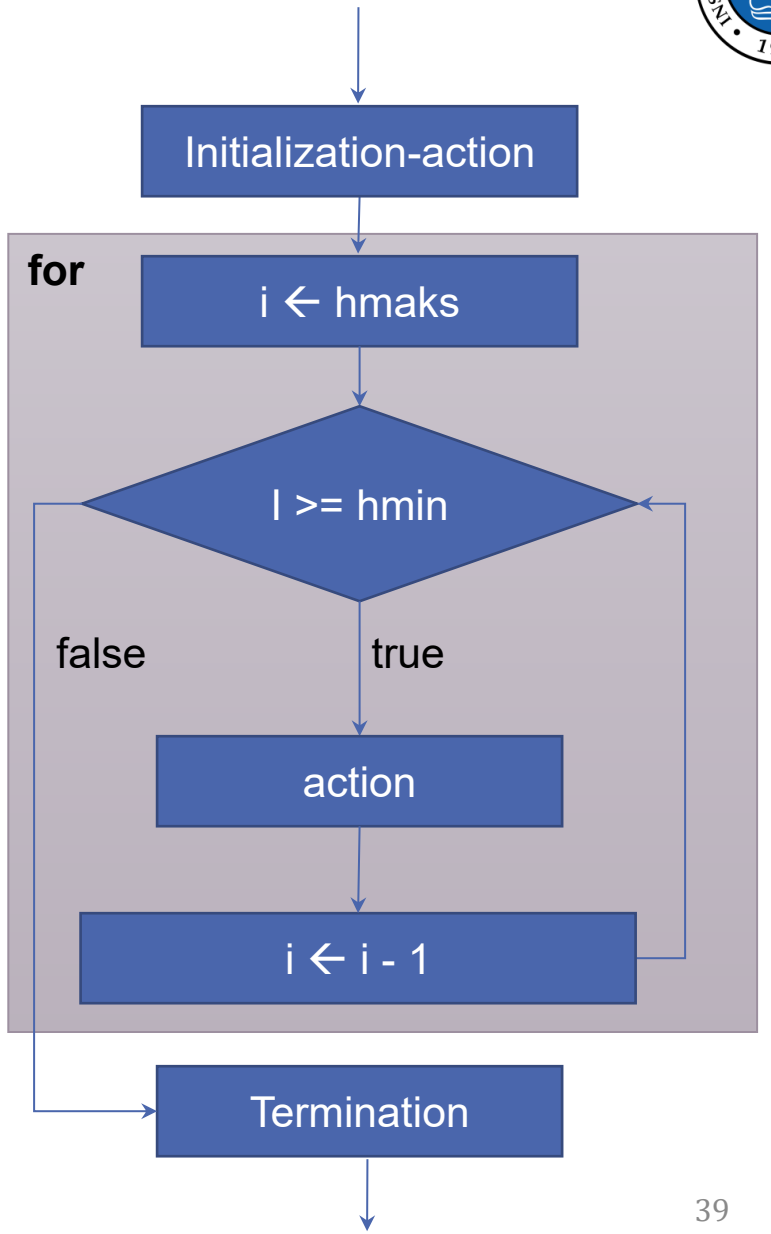
```
Initialization-action
for i in range(hmax,hmin-1,-1):
    action
Termination
```

i is a counter variable (can be replaced by another variable)
hmax = value of i at the beginning of the loop; *hmin* = last i value processed;
The value of i when exiting the loop is hmin-1
Every time it repeats, i is decremented (minus 1)

pseudocode

```
Initialization-action
i traversal [hmax..hmin]
   action
Termination
```

flowchart



Looping based on repetition-condition at the beginning [while]

- **actions** are carried out as long as **repetition-condition** is still satisfied (true)
- This type of repetition has the potential to give rise to **"empty" action** (action never performed at all)
  - Because in the first test of the repetition-condition, it returns false, so it never visits and executes repetition-body and immediately exits the loop

Based on repetition-condition at the beginning

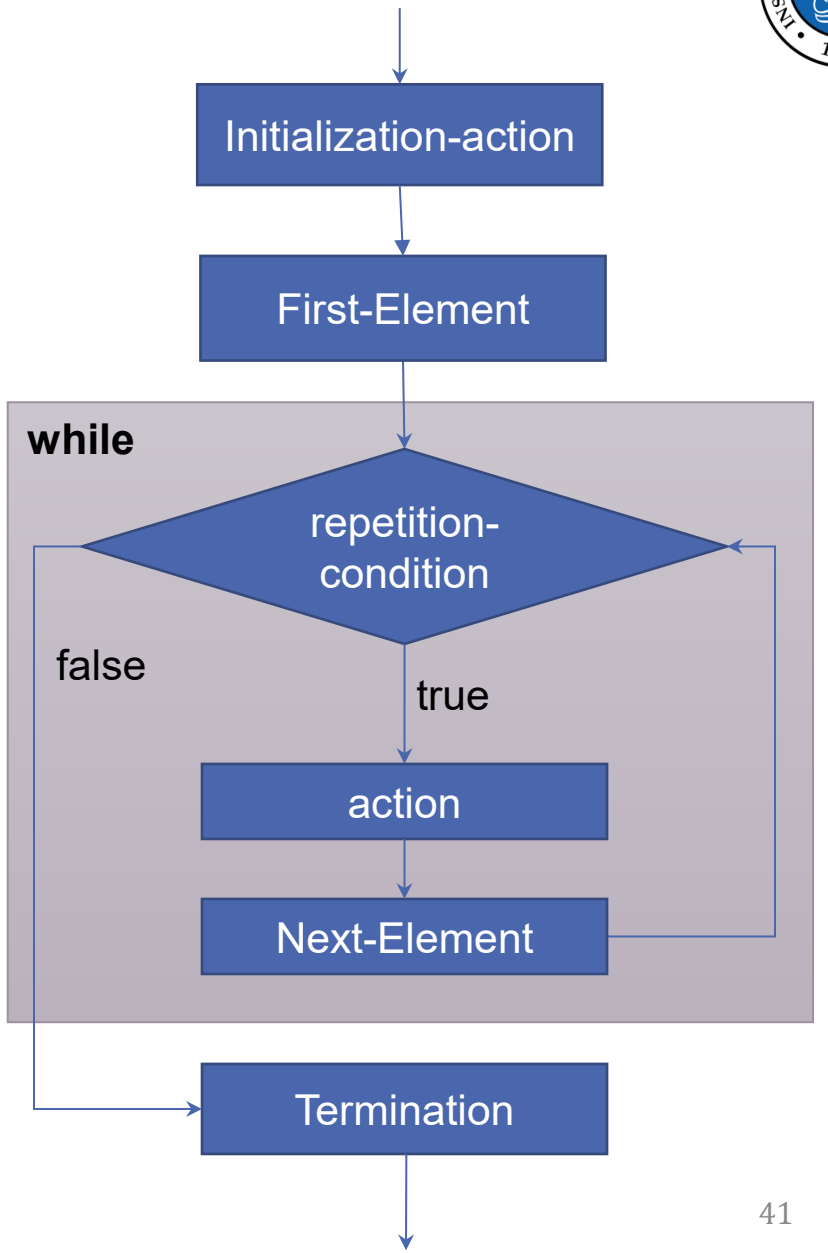
Python

```
Initialization-action
First-Element
while (repetition-condition):
    action
    Next-Element
# repetition-condition=false
Termination
```

pseudocode

```
Initialization-action
First-Element
while (repetition-condition) do
    action
    Next-Element
{ repetition-condition = false }
Termination
```

flowchart



Example 1. while

```
# Program AddUpNumbers
# Calculate 1+2+3+...+N. Assumption: N > 0

# DECLARATIONS
# N : int
# i, sum : int

# ALGORITHM
N = int(input())      # initialization
sum = 0               # initialization
i = 1                 # First-Element
while (i <= N):       # repetition-condition
      print(i)        # action
      sum = sum + i   # action
      i = i + 1       # Next-Element
# i > N
print(sum)            # Termination
```

Example 2

- Write a program that accepts input of 10 integer numbers (from the keyboard) and prints to the screen the total number of the 10 integers.
- Example:

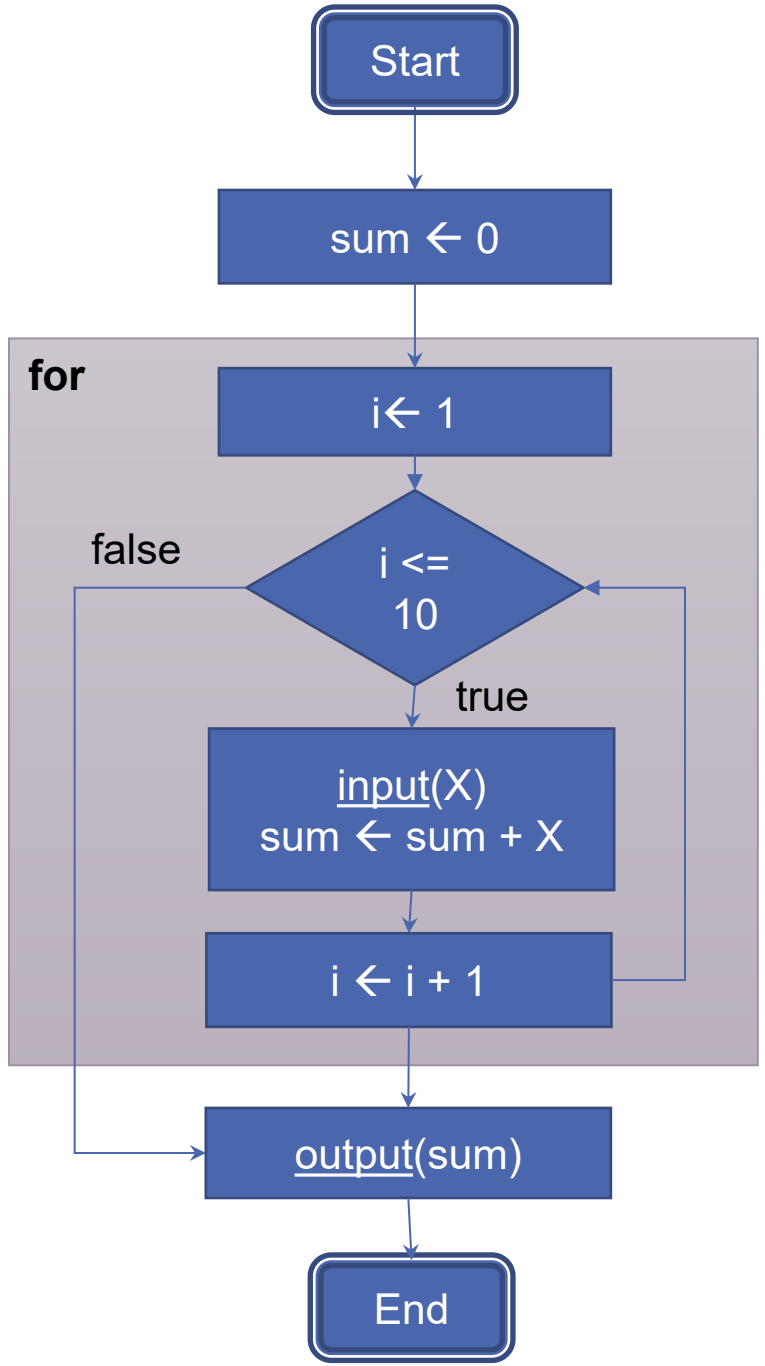
Input	Output on screen
2 1 0 -9 7 13 2 2 1 -1	18

Example 2. for Pseudocode+Flowchart

pseudocode

```
Sum ← 0  { Initialization }
i traversal [1..10]
    input(X)        { action }
    sum ← sum + X  { action }
output(sum)  { Termination }
```

flowchart



Example 2. for Python

```
# Program AddUp10Number
# Accepts input of 10 integers and
# print the total number of the 10 integers

# DECLARATIONS
# N, i, sum : int

# ALGORITHM
sum = 0                          # initialization

for i in range(1, 11):
    N = int(input())             # action
    sum = sum + N                # action

print(sum)                       # termination
```

Example 2. Discussions

- It is most appropriate to use **“for”**:
  - Because the number of times the action must be repeated is known for certain, namely 10x, meaning the range of counter prices for repetition is known for certain, namely from 1..10 (the last value of the counter when it exits the loop = 11)
- It is not appropriate to use **“while”** because there is no possibility of an "empty" case
  - “while” is more appropriate to use if there is a possibility that the action will never be carried out at all (empty case), in this case, the action will definitely be carried out, at least once.

Example 3

- Write a program that reads a number of integer numbers from the keyboard until the user enters the number -999 (the number -999 does not include the processed number).
- Print how many numbers were entered, the total value, and the average of all the numbers
- If the first input says -999, then write the message "No data processed"

- Hint: Use a **while** loop

No	Input	Output
1	**-1** **12** **-6** **10** **2** **-999**	Number of numbers = **5** Total number = **17** Average = **3.40**
2	**-999**	**No data is processed**

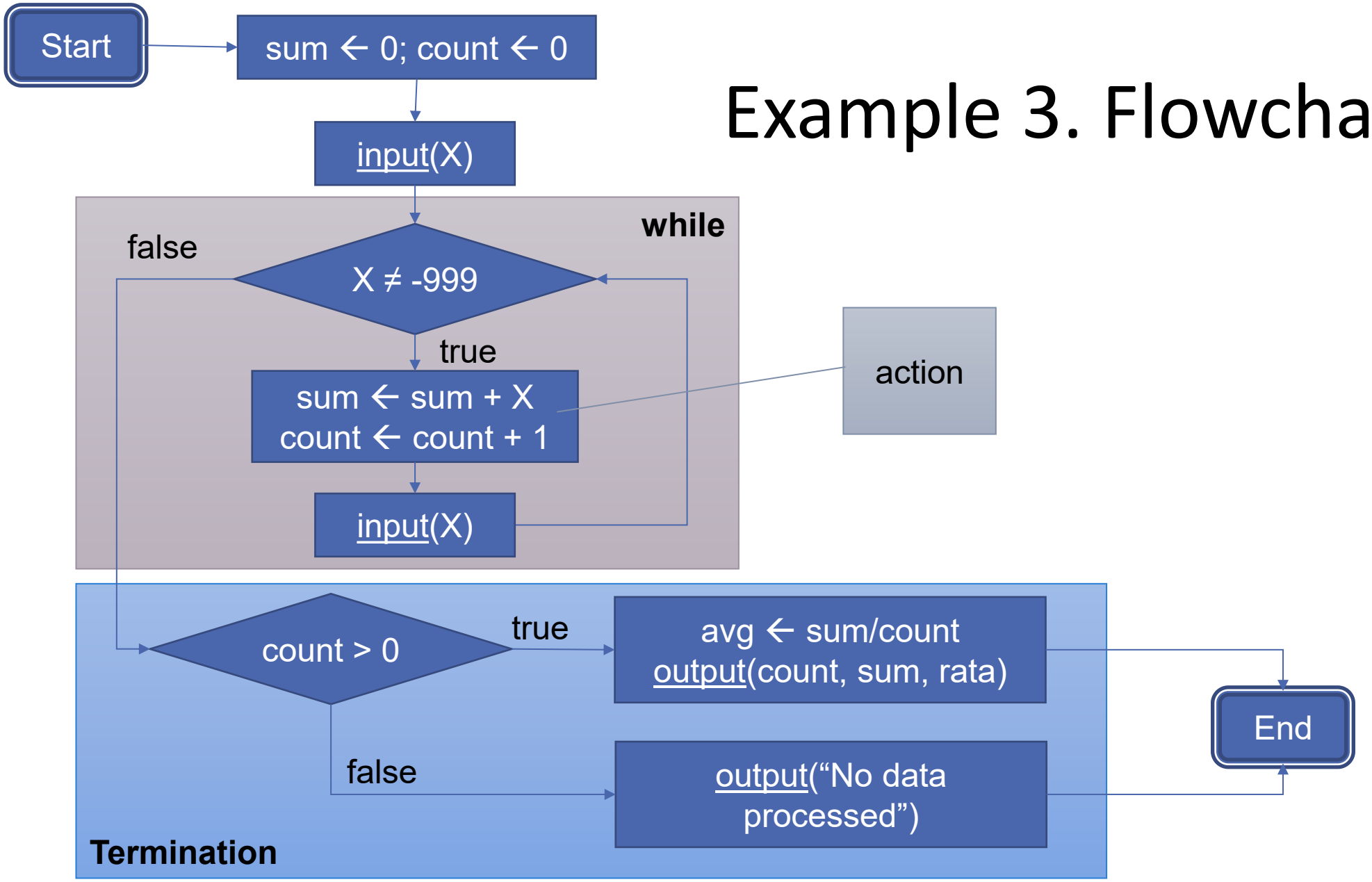
Example 3. Pseudocode

```
sum ← 0; count ← 0  { Initialization }

input(X)     { First-Elmt }
{ Looping using while-do }
while (X ≠ -999) do
   sum ← sum + X
   count ← count + 1
   input(X)
{ X = -999 }

{ Termination }
if (count > 0) then
   avg ← sum/count
   output(count,sum,rata)
else { count = 0 }
   output("No data processed")
```

Example 3. Flowchart



Example 3. Python

```
# Program AvgNumbers
# Accepts input of an integer number until the user enters -999
# and displays the number of numbers, the total, and the average

# DECLARATIONS
# X, count, sum : int
# avg : float

# ALGORITHM
sum = 0; count = 0      # initialization

X = int(input())        # First-Elmt
while (X != -999):
    count = count + 1   # action
    sum = sum + X
    X = int(input())    # Next-Elmt
# X = -999

# Termination
if (count > 0):
    print("Number of numbers = " + str(count))
    print("Total number = " + str(sum))
    avg = sum/count
    print("Average = " + str(rata))
else:
    print ("No data processed")
```

Example 3. Discussions

- Repetition using "**while**" is most appropriate because:
  - There is a possibility that the action is never carried out at all (empty case), namely if the X value that the user first enters is -999 (see Example 2)
- "**for**" is not appropriate to use because it does not have a defined value range